

Session Akışı

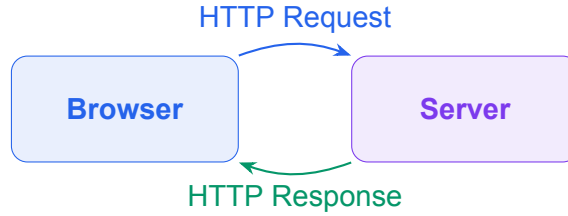
Süre	Konu	Anahtar Kavramlar
0–15 dk	Web Nasıl Çalışır?	Browser, Server, HTTP, Request/Response
15–35 dk	HTML Temelleri	Etiketler, Semantic HTML, Form öğeleri
35–70 dk	CSS Temelleri	Box Model, Display, Flexbox
70–90 dk	Pratik: Card UI	Flex layout, gerçek bileşen

1 🌐 Web Nasıl Çalışır?

🕒 0–15 dakika

1.1 Browser – Server İlişkisi

Web uygulamaları iki taraf arasındaki bir iletişimdir:



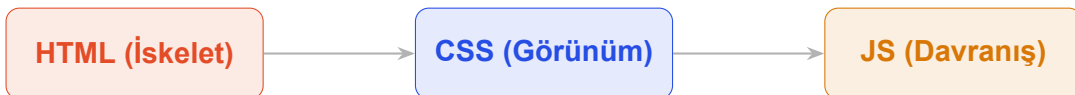
Temel Kavramlar

- **Browser (Tarayıcı):** Chrome, Firefox, Safari. HTML/CSS/JS'i okuyup ekrana çizer.
- **Server (Sunucu):** İstekleri alan, cevap dönen bilgisayar/servis.
- **HTTP:** İkisi arasındaki konuşma dili (protokol).
- **URL:** Kaynak adresi — `https://ornek.com/sayfa`

1.2 Request / Response Döngüsü

1. Kullanıcı adres çubuğuna URL yazar ve Enter'a basar.
2. Browser sunucuya **HTTP Request** gönderir.
3. Sunucu isteği işler ve **HTTP Response** döner (HTML dosyası vb.).
4. Browser aldığı HTML'i okur, CSS ile renklendirir, JS ile canlandırır.

1.3 HTML – CSS – JS Akışı



💡 Analoji

Bir binanın **iskeleti** HTML, **boyası ve dekorasyonu** CSS, **asansörü ve otomatik kapıları** JavaScript'tir.

2 </> HTML Temelleri

🕒 15–35 dakika

2.1 Temel Yapı

Minimal HTML Belgesi

```
<!DOCTYPE html>
<html lang="tr">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial
      -scale=1.0" />
    <title>My Page</title>
  </head>
  <body>
    <h1>Hello World!</h1>
    <p>My first HTML page.</p>
  </body>
</html>
```

⚠ Dikkat

Her açılan etiketin kapatılması gerekir: <p> açılır, </p> kapatılır. Bazı etiketler self-closing'dir: <meta />, , <input />

2.2 Semantic (Anlamlı) Etiketler

Semantic etiketler içeriğin *ne olduğunu* söyler; okunabilirlik ve SEO'yu iyileştirir.

Etiket	Anlamı
<header>	Sayfa veya bölüm başlığı
<nav>	Navigasyon/menü alanı
<main>	Sayfanın ana içeriği (sayfa başına 1 tane)
<section>	Tematik bir içerik bölümü
<article>	Bağımsız, paylaşılabilir içerik (blog yazısı vb.)
<aside>	Yan bilgi, kenar çubuğu
<footer>	Sayfa veya bölüm altlığı

Semantic Layout Örneği

```
<body>
  <header>
    <nav>
      <a href="/">Home</a>
      <a href="/about">About</a>
    </nav>
  </header>

  <main>
    <section>
      <h2>Featured Products</h2>
      <article>
        <h3>Product Name</h3>
        <p>Description text...</p>
      </article>
    </section>
  </main>

  <footer>
    <p>2026 Company Name</p>
  </footer>
</body>
```

2.3 Form Elementleri

Form Örneği

```
<form>
  <!-- Text input -->
  <label for="fullName">Full name:</label>
  <input type="text" id="fullName" name="fullName" placeholder
    ="Enter your name" />

  <!-- Email -->
  <label for="email">Email:</label>
  <input type="email" id="email" name="email" />

  <!-- Buttons -->
  <button type="submit">Submit</button>
  <button type="button">Cancel</button>
</form>
```

💡 label + id bağlantısı

`<label for="ad">` ile `<input id="ad">` eşleşince, etikete tıklamak imleci input'a taşır — erişilebilirlik için kritiktir.

3 🛠️ CSS Temelleri

🕒 35–70 dakika

3.1 CSS Nasıl Bağlanır?

CSS Bağlama Yöntemleri

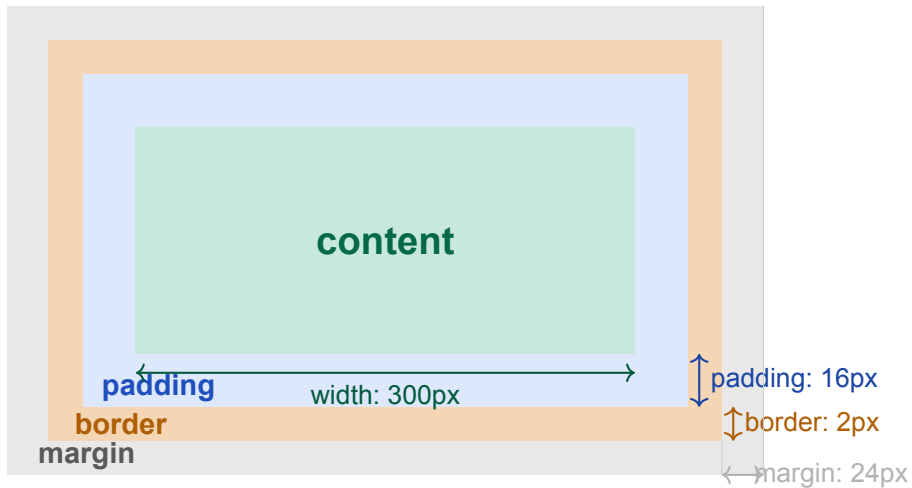
```
<!-- 1. External file (recommended) -->
<link rel="stylesheet" href="style.css" />

<!-- 2. Internal <style> tag -->
<style>
  body { margin: 0; }
</style>

<!-- 3. Inline style (avoid) -->
<p style="color: red;">Text</p>
```

3.2 Box Model

Her HTML elemanı aslında bir **kutu**dur ve dört katmandan oluşur:



```
.kutu {
  width: 300px;           /* content width */
  padding: 16px;          /* inner space (all 4 sides) */
  border: 2px solid #ccc; /* border width */
  margin: 24px;           /* outer space (all 4 sides) */

  /* padding + border included in total width */
  box-sizing: border-box;
}
```

3.3 Display Özellikleri

`display` özelliği, bir elemanın sayfada nasıl konumlandırılacağını belirler.

div

p

h1

Tam satırı kaplar, alt alta sıralanır.

```
div, p, h1, h2, section,
article, header, footer {
  display: block; /* default */
}
```

span

strong

a

em

Yan yana, içerik kadar yer kaplar.

```
span, a, strong, em {
  display: inline; /* default */
  /* width/height not applicable!
  */
}
```

.konteyner

A B C

ana eksen

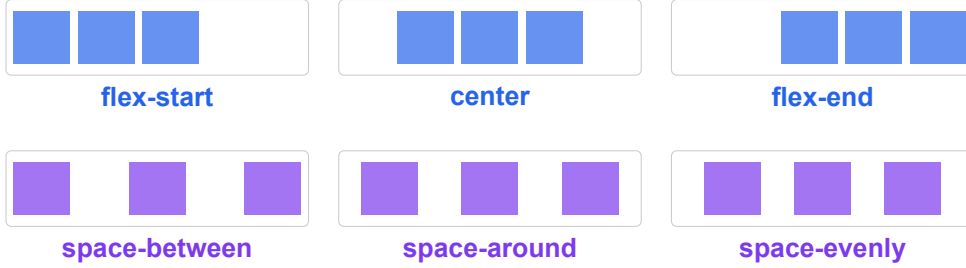
```
.konteyner {
  display: flex;
  /* children placed side by side
  and auto-aligned */
}
```

3.4 Flexbox

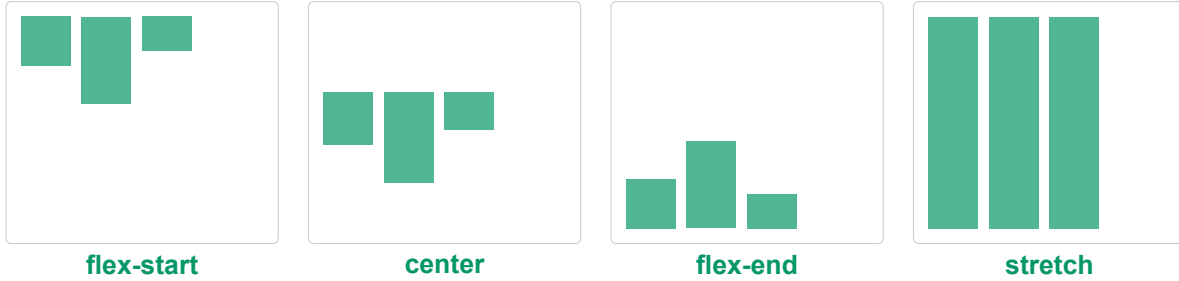
Flexbox Nedir?

Elemanları tek eksenle (satır veya sütun) hizalamak için tasarlanmış CSS düzen sistemi. Modern arayüzlerin temel taşıdır.

3.4.1 justify-content — Ana Eksen Hizalaması



3.4.2 align-items — Çapraz Eksen Hizalaması



Flexbox Örnekleri

```
/* Center both horizontally and vertically */
.container {
  display: flex;
  justify-content: center; /* horizontal (main axis) */
  align-items: center; /* vertical (cross axis) */
  height: 100vh;
}

/* Spread cards side by side */
.card-list {
  display: flex;
  justify-content: space-between;
  gap: 16px;
  flex-wrap: wrap; /* wrap to next line */
}

/* Navbar: logo left, links right */
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0 24px;
}
```

4 Pratik: Card UI Oluşturma

 70–90 dakika

☰ Pratik Hedef

Flexbox kullanarak üç kart içeren bir ürün listesi sayfası oluşturun. Kartlar:

- Görsel (placeholder), başlık, açıklama ve buton içerecek
- Yan yana sıralanacak
- Hover'da hafif gölgelenecek

Adım 1 — HTML Yapısı

index.html

```
<!DOCTYPE html>
<html lang="tr">
  <head>
    <meta charset="UTF-8" />
    <title>Product Cards</title>
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>

    <header class="site-header">
      <h1>Our Products</h1>
    </header>

    <main class="card-grid">

      <!-- Each card repeats the same structure -->
      <article class="card">
        
        <div class="card-body">
          <h2 class="card-title">Product Title</h2>
          <p class="card-desc">Short product description.</p>
          <button class="btn">View</button>
        </div>
      </article>

      <!-- <article class="card"> ... </article>  x2 more -->

    </main>

  </body>
</html>
```

Adım 2 — CSS Stilleri

style.css

```
* { box-sizing: border-box; margin: 0; padding: 0; }

body { font-family: sans-serif; background: #f1f5f9; color: #1e293b; }

.site-header {
  background: #2563eb;
  color: white;
  padding: 24px;
  text-align: center;
}

.card-grid {
  display: flex;
  justify-content: center;
  gap: 24px;
  flex-wrap: wrap;
  padding: 40px 24px;
}

.card {
  background: white;
  border-radius: 12px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.08);
  width: 300px;
  overflow: hidden;
  display: flex;
  flex-direction: column;
  transition: box-shadow 0.2s ease, transform 0.2s ease;
}

.card:hover {
  box-shadow: 0 8px 24px rgba(0,0,0,0.15);
  transform: translateY(-4px);
}

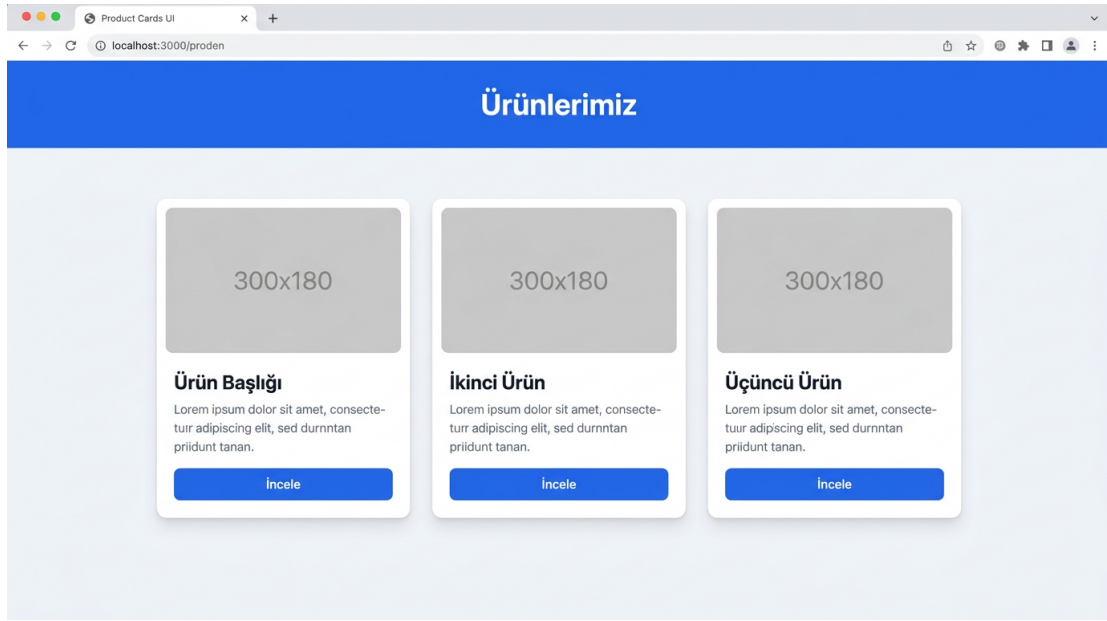
.card img { width: 100%; display: block; }

.card-body {
  padding: 20px;
  display: flex;
  flex-direction: column;
  gap: 12px;
  flex: 1;
}

.card-title { font-size: 1.1rem; font-weight: 700; }
.card-desc { font-size: 0.9rem; color: #64748b; flex: 1; }

.btn {
  background: #2563eb;
  color: white;
  border: none;
  border-radius: 8px;
  padding: 10px 20px;
```

Tarayıcı Çıktısı



Pratik kodunun tarayıcıdaki görünümü: üç kart yan yana, hover efektiyle

5 Özet ve Sonraki Session

5.1 Bu Session'da Öğrendiklerimiz

- ✓ Web nasıl çalışır: Browser–Server, HTTP, Request/Response
- ✓ HTML iskelet yapısı: `<!DOCTYPE>`, `<head>`, `<body>`
- ✓ Semantic etiketler: `header`, `nav`, `main`, `section`, `footer`
- ✓ Form öğeleri: `input`, `label`, `button`
- ✓ CSS Box Model: `content` / `padding` / `border` / `margin`
- ✓ `display`: `block`, `inline`, `flex` farkları
- ✓ Flexbox: `justify-content`, `align-items`, `gap`, `flex-wrap`
- ✓ Pratik: Card UI — gerçek bir bileşen sıfırdan yazdık

5.2 Sıkça Sorulan Sorular

SSS

S: `div` ile `section` ne zaman kullanılır?

C: `<section>` tematik bir içerik grubuysa, `<div>` yalnızca stil/layout için bir kapsayıcıysa kullanılır.

S: `padding` mi `margin` mi?

C: Elemanın iç boşluğu için `padding`, dış boşluğu için `margin`.

S: Flexbox mu Grid mi?

C: Tek boyutlu (satır veya sütun) düzen için Flexbox, iki boyutlu (satır ve sütun) için CSS Grid kullanılır.

5.3 Sonraki Session — Session 2: Modern JavaScript

→ Session 2 (29 Nisan Çarşamba)

React öncesi için ihtiyaç duyacağımız temel JavaScript konuları:

- **let / const**, arrow functions, template literals
- **Destructuring**, spread/rest operatörleri
- **Array methods**: `map`, `filter`, `reduce`
- **async/await** ve `fetch` API ile veri çekme